Robotic Navigation and Exploration

HW2: Kinematic Model and Path Tracking Control

Min-Chun Hu anitahu@cs.nthu.edu.tw
CS, NTHU

Homework2 - Report

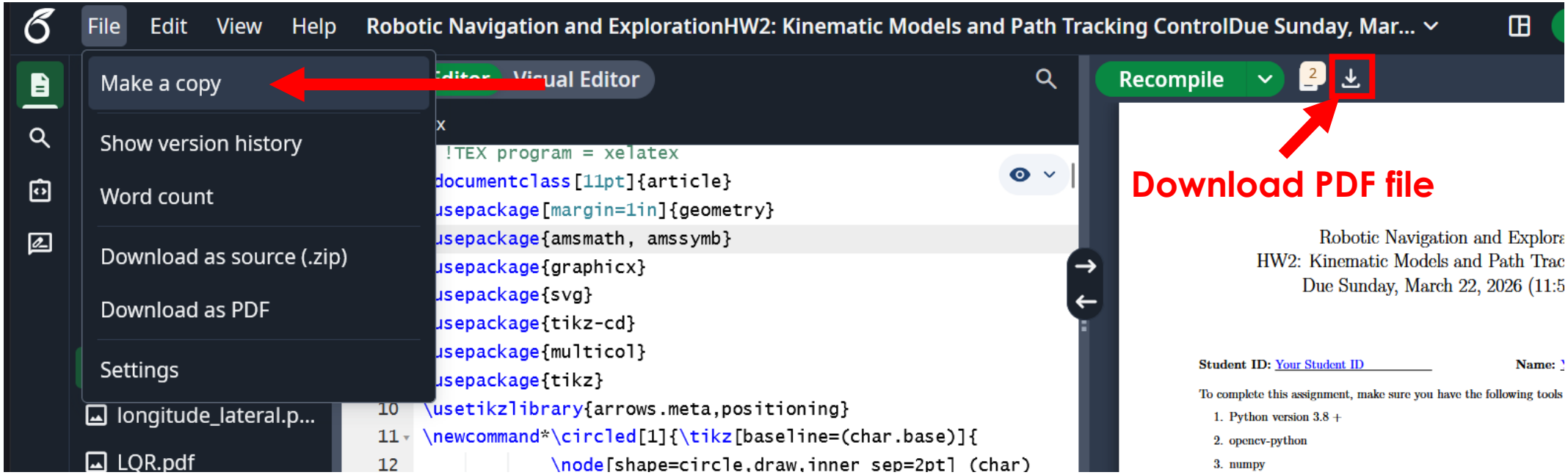
- All tasks required in this homework are already listed in the **report file**, just follow the instructions, do the questions one-by-one and you can finish the homework!
- In the report file,

  : **Writing Part**, keep your answer in blue.

  : **Programming Part**, test command is provided for each programming question.

Homework2 - Report

- Log in/Sign up Overleaf
- Go to link: https://www.overleaf.com/read/xjnwwpbhzvhk#c13aac
- Make a copy to your account



Homework2

- This slide is to review the flow of **(simplified) trajectory planning** and **path tracking** and help you get familiar with code structure.
  - **Trajectory Planning**: Generate the **trajectory** $\{[\boldsymbol{x_0}, \boldsymbol{y_0}, \boldsymbol{\theta_0}, \boldsymbol{v_0}], [\boldsymbol{x_1}, \boldsymbol{y_1}, \boldsymbol{\theta_1}, \boldsymbol{v_1}], \ldots\}$, which consists of waypoints with motion information
  - **Path Tracking:** Determine the **control input** $\boldsymbol{u}$ for the next timestamp based on the **current state** $\boldsymbol{\zeta}$ and **waypoints**

Trajectory Planning

Sparse Waypoints (HW1)

Cubic Curve Interpolation

Dense & Smooth Path

Velocity Profiling

Reference Speed

Waypoint Sampling

Trajectory

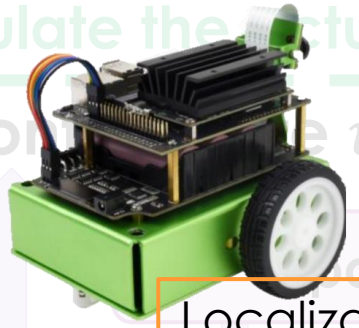
$\{[x_0, y_0, \theta_0, v_0], [x_1, y_1, \theta_1, v_1], \ldots.\}$

Path Tracking

Simulate the actual vehicle motion

Control State u_{t-1}

Update Simulator



mobile robot

Localization module

State $\zeta_t = [x_t, y_t, \theta_t]^T$

Longitudinal Control ↕

Lateral Control ↔

Control State u_t

Trajectory Planning

Sparse Waypoints (HW1)

Cubic Curve Interpolation

Dense & Smooth Path

Velocity Profiling

Reference Speed

Waypoint Sampling

Trajectory

$\{[x_0, y_0, \theta_0, v_0], [x_1, y_1, \theta_1, v_1], \dots.\}$

Path Tracking

Simulate the actual vehicle motion

Control State u_{t-1}

Update Simulator

State $\zeta_t = [x_t, y_t, \theta_t]^T$

Longitudinal Control ↕

Lateral Control ↔

Control State u_t

Trajectory Planning

Sparse Waypoints (HW1)

Cubic Curve Interpolation

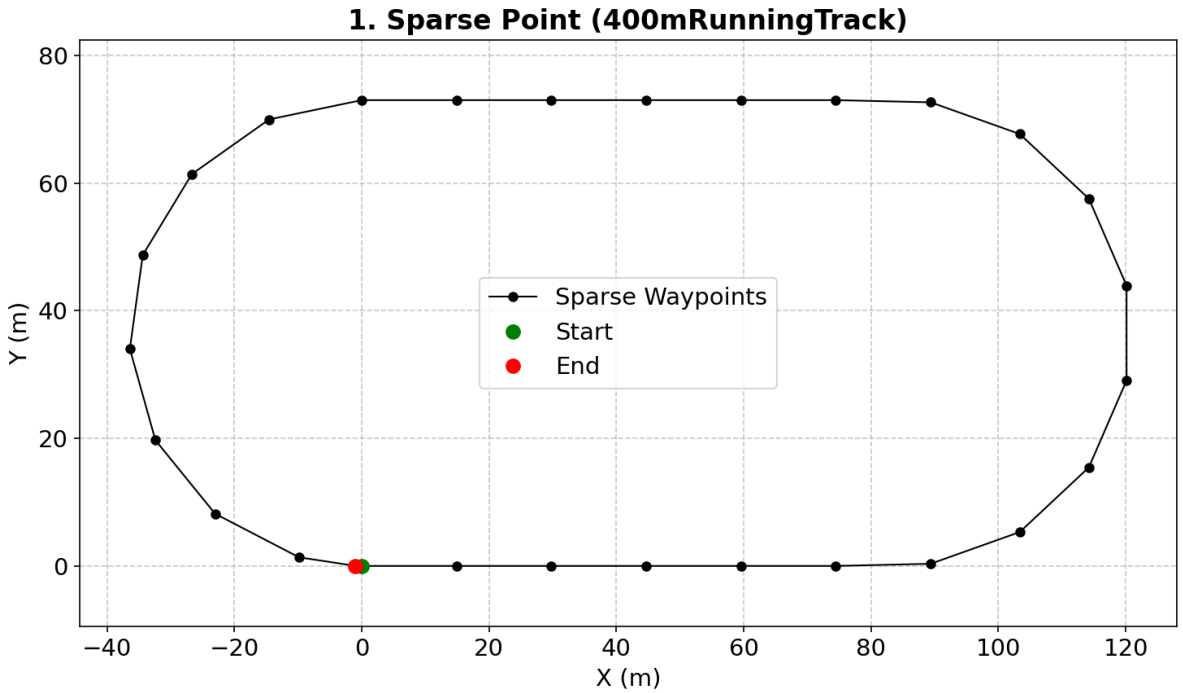
Dense & Smooth Path

Velocity Profiling

Reference Speed

Waypoint Sampling

Trajectory



The raw waypoints from HW1 are not sufficient for the mobile robot to navigate smoothly

Trajectory Planning

Sparse Waypoints (HW1)

Cubic Curve Interpolation

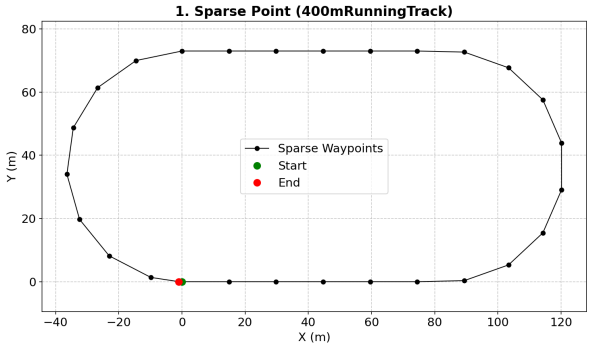
Dense & Smooth Path

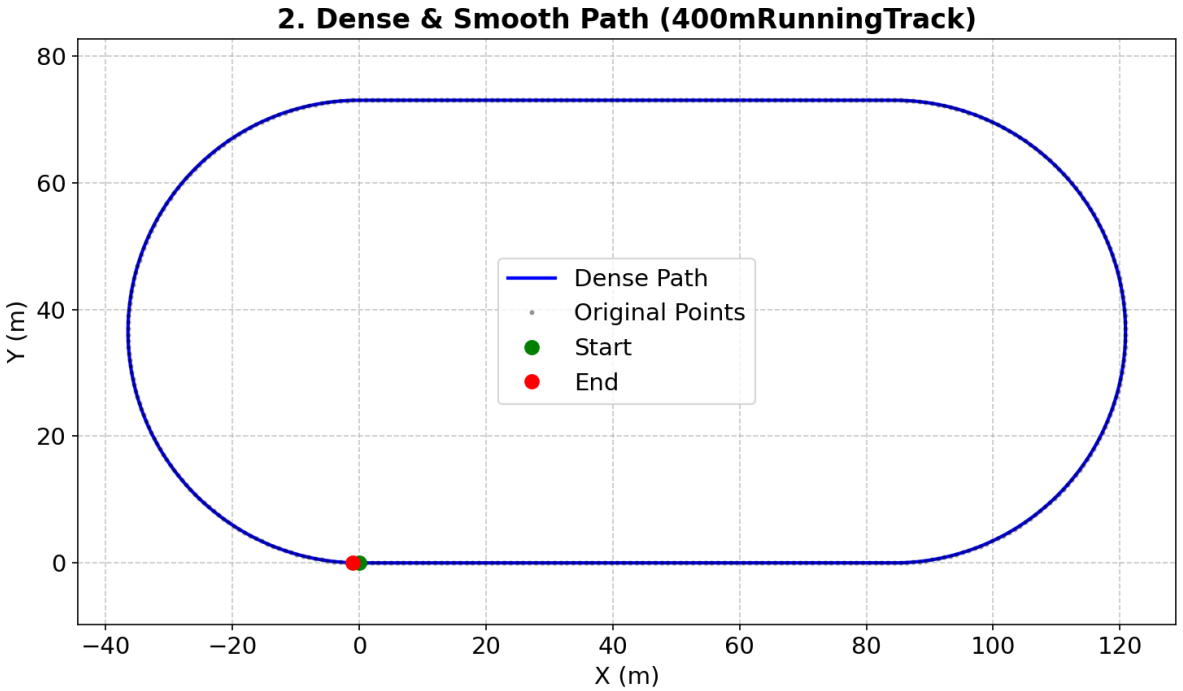
Velocity Profiling

Reference Speed

Waypoint Sampling

Trajectory





Trajectory Planning

Sparse Waypoints (HW1)

Cubic Curve Interpolation

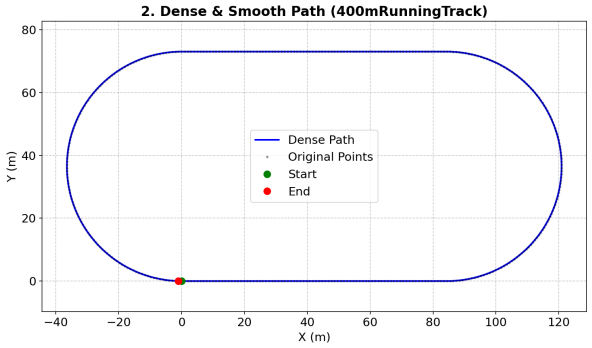
Dense & Smooth Path

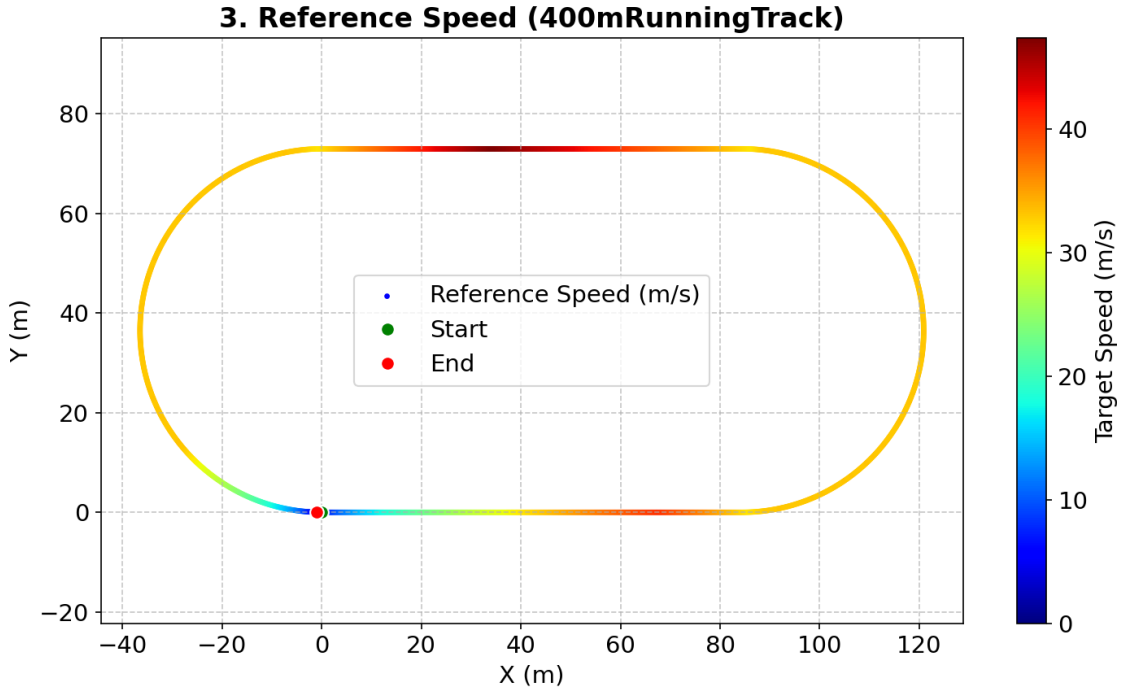
Velocity Profiling

Reference Speed

Waypoint Sampling

Trajectory





Velocity Profiling → Reference Speed v_{ref}

1. Curvature κ
2. Speed limit: Consider maximum speed $\boldsymbol{v_{max}}$ and lateral acceleration $\boldsymbol{a_{lat}}$

$$v_{\text{limit}} = \min\left(v_{\text{max}}, \sqrt{\frac{a_{\text{lat}}}{\kappa}}\right)$$

3. Longitudinal smoothing:
   1. Forward pass: limit acceleration $\boldsymbol{a_{\text{acc}}}$

      $$v_i^2 \leq v_{i-1}^2 + 2 \cdot a_{\text{acc}} \cdot ds$$

   2. Backward pass: limit deceleration $\boldsymbol{a_{\text{dec}}}$

      $$v_i^2 \leq v_{i+1}^2 + 2 \cdot a_{\text{dec}} \cdot ds$$

   ds is the distance between point I and point i+1

? Where to set

$v_{\text{ref}}[0]$ = 5 (start), and

$v_{\text{ref}}[-1]$ = 0 (end)

Trajectory Planning

Sparse Waypoints (HW1)

Cubic Curve Interpolation

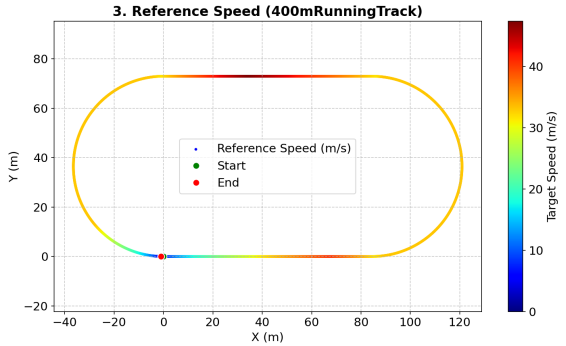
Dense & Smooth Path

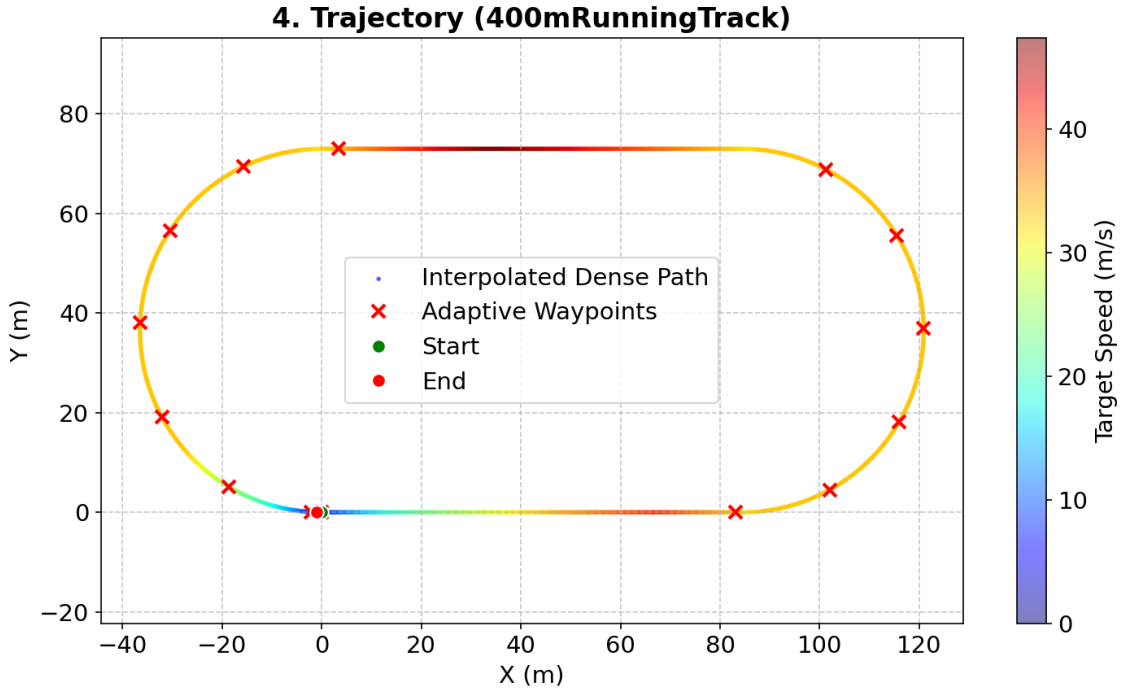
Velocity Profiling

Reference Speed

Waypoint Sampling

Trajectory





Trajectory Planning

Sparse Waypoints (HW1)

Cubic Curve Interpolation

Dense & Smooth Path

Velocity Profiling

Reference Speed

Waypoint Sampling

Trajectory

$\{[x_0, y_0, \theta_0, v_0], [x_1, y_1, \theta_1, v_1], \dots\}$

Path Tracking

Simulate the actual vehicle motion

Control State u_{t-1}

Update Simulator

State $\zeta_t = [x_t, y_t, \theta_t]^T$

Longitudinal Control ↕

Lateral Control ↔

Control State u_t

Path Tracking

Control State u_{t-1}

Update Simulator

State $\zeta_t = [x_t, y_t, \theta_t]^T$

Longitudinal Control ↕

Lateral Control ↔

Control State u_t

Path Tracking

Control State u_{t-1}

Update Simulator

feasible control u'

Kinematic Models

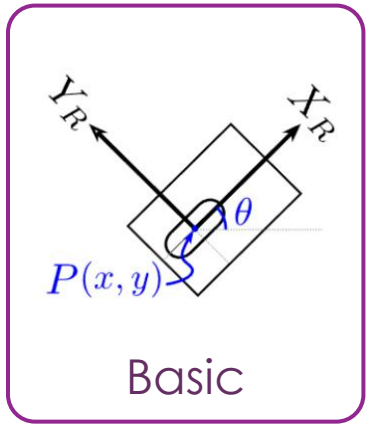
State $\zeta_t = [x_t, y_t, \theta_t]^T$

Define the state transition $\zeta = [x, y, \theta]^T$ based on the control input u'.

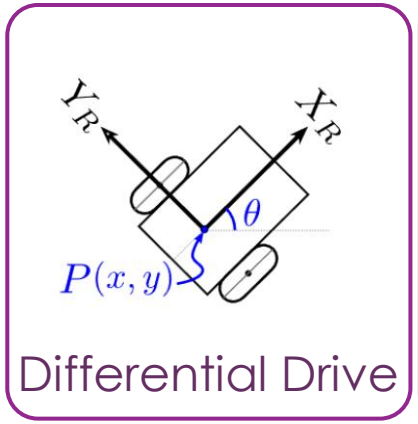
Longitudinal Control ↕

Lateral Control ↔

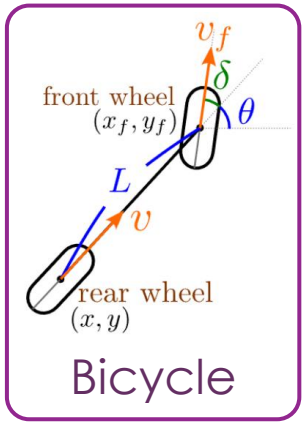
Control State u_t



Basic



Differential Drive



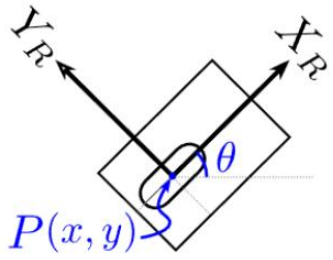
Bicycle

Feasible control u'

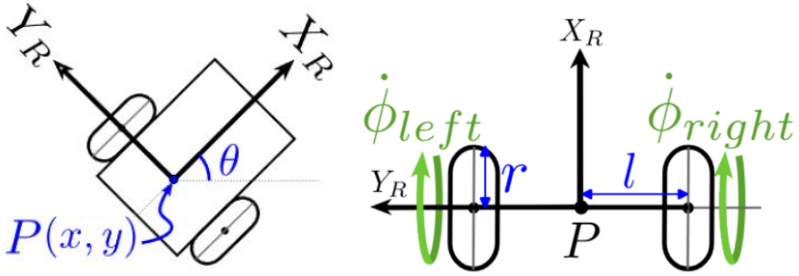
State ζ

Kinematic Models

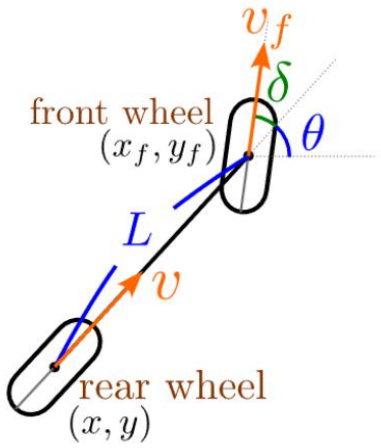
Define the **state** transition $\boldsymbol{\zeta} = [\boldsymbol{x}, \boldsymbol{y}, \boldsymbol{\theta}]^{\boldsymbol{T}}$ based on the **control input** $\boldsymbol{u}'$.



Basic



Differential Drive



Bicycle

$$u = [v, \omega]^T$$

$$\dot{\zeta} = \begin{bmatrix} v\cos\theta \\ v\sin\theta \\ \omega \end{bmatrix}$$

$$u = [\dot{\phi}_{left}, \dot{\phi}_{right}]^T$$

$$\dot{\zeta} = \begin{bmatrix} \frac{r}{2}(\dot{\phi}_{\text{left}} + \dot{\phi}_{\text{right}})\cos\theta \\ \frac{r}{2}(\dot{\phi}_{\text{left}} + \dot{\phi}_{\text{right}})\sin\theta \\ \frac{r}{l}(\dot{\phi}_{\text{right}} - \dot{\phi}_{\text{left}}) \end{bmatrix}$$

$$u = [a, \delta]^T$$

$$\dot{\zeta} = \begin{bmatrix} v\cos\theta \\ v\sin\theta \\ \frac{v}{L}\tan\delta \end{bmatrix}$$

Path Tracking

Control State u_{t-1}

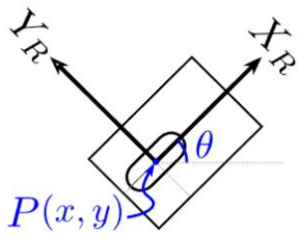
Update Simulator

State $\zeta_t = [x_t, y_t, \theta_t]^T$

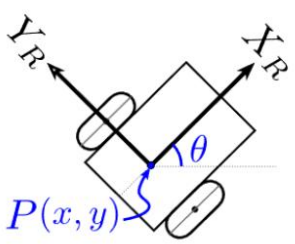
Longitudinal Control ↕

Lateral Control ↔

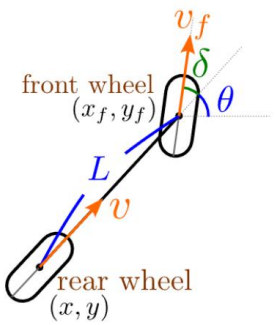
Control State u_t



Basic



Differential Drive



Bicycle

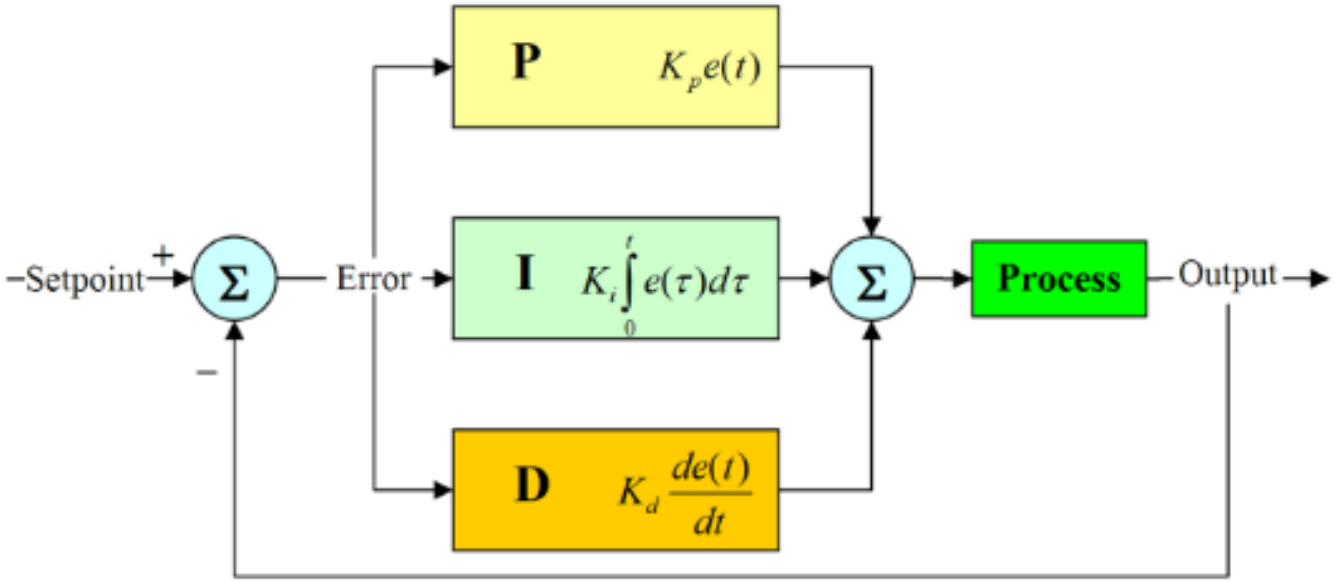
Directly use v_{ref}	PID to track v_{ref}
PID Pure Pursuit LQR	PID Pure Pursuit Stanley LQR

*Basic and Differential Drive share the same controller file

PID Control

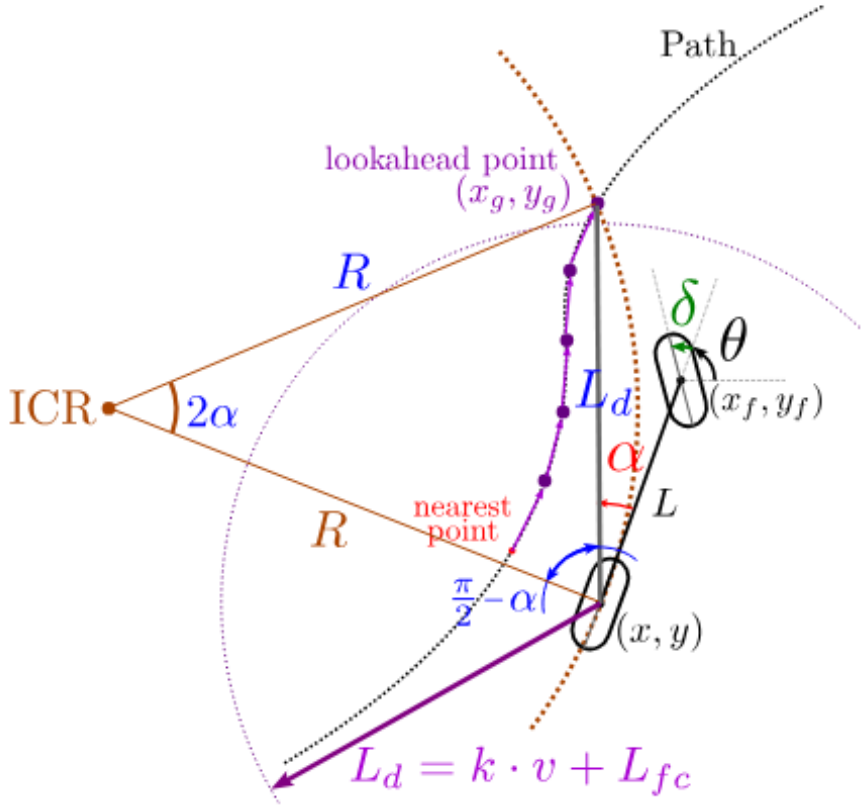
PID formula:

$$e(t) = v_{\mathbf{ref}}(t) - v(t), \qquad u(t) = a = K_p\, e(t) + K_i \int_0^t e(\tau)\, d\tau + K_d \frac{de(t)}{dt}.$$



Pure-Pursuit Control

The Pure Pursuit algorithm calculates the steering angle δ needed to guide the vehicle toward a "look-ahead" point on the path.



Pure Pursuit Algorithm

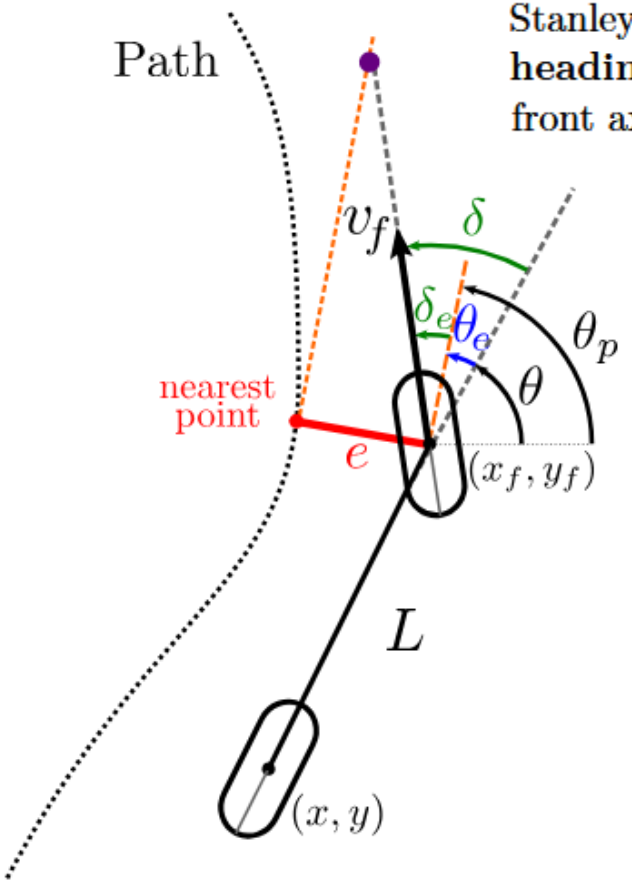
1. Find the nearest point on the path.
2. Find the first point on the path that is at least L_d away.

$$L_d = k \times v + L_{fc}$$

3. Calculate the steering angle δ to move the vehicle toward the "look-ahead" point.

$$\delta = \arctan\left(\frac{2L\sin(\alpha)}{L_d}\right)$$

Stanley Control



Stanley control is a path-tracking algorithm that determines the steering angle by combining the **heading error** relative to the path with the **cross-track error** measured from the center of the front axle to the nearest point on the path.

Stanley Control Algorithm

1. Find the nearest point on the path.
2. Calculate the heading error θ_e and the cross-track error δ_e.

$$\theta_e = \theta_{path} - \theta$$

$$|e| = \sqrt{(x - x_{path})^2 + (y - y_{path})^2}$$

3. Calculate the steering angle δ to move the vehicle toward the "look-ahead" point.

$$\delta = \theta_e + \delta_e = \theta_e + \arctan\left(\frac{-ke}{v_f}\right) \qquad (4)$$

LQR Control

- Follow these steps:
  - Construct the matrix A, B, X of the linear approximation model.

$$
\begin{aligned}
\mathbf{x}_{k+1} &= A\mathbf{x}_k + B\mathbf{u}_k \\
&= \begin{bmatrix} e_k \\ \theta_{e,k} \end{bmatrix} + \begin{bmatrix} \dot{e}_k \\ \dot{\theta}_{e,k} \end{bmatrix} \Delta t \\
&= \begin{bmatrix} 1 & -v \cdot \Delta t \\ 0 & 1 \end{bmatrix} \begin{bmatrix} e_k \\ \theta_{e,k} \end{bmatrix} + \begin{bmatrix} 0 \\ -\frac{v}{L}\Delta t \end{bmatrix} \begin{bmatrix} \delta_k \end{bmatrix}
\end{aligned}
$$

  - Solve DARE and get the matrix P of the value function

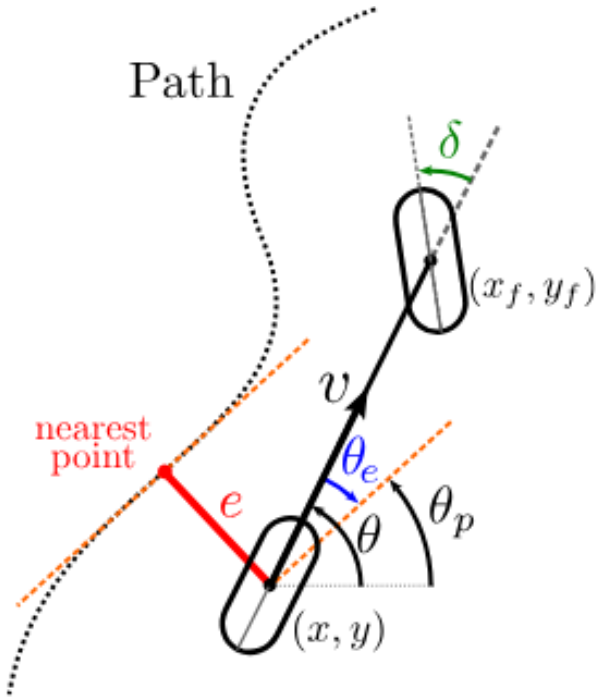
$$P = A^T P A - (A^T P B)(R + B^T P B)^{-1}(B^T P A) + Q$$

  - Compute the optimal control:

$$\mathbf{u}_k = -K\mathbf{x}_k \qquad K = (R + B^T P B)^{-1}(B^T P A)$$

LQR Control

- Derivation of linear approximation model:
- Assume state is $\mathbf{x} = [e, \theta_e]^T$ where e is the cross-track error and θ_e is the heading error, and the control input is $\mathbf{u} = [\delta]^T$. For the bicycle model, we have the following kinematic equations:



$$\dot{e} = -v\sin(\theta_e)$$

$$\dot{\theta}_e = -\frac{v}{L}\tan(\delta)$$

In LQR, we assume that θ_e and δ are small, so we can approximate $\sin(\theta_e) \approx \theta_e$ and $\tan(\delta) \approx \delta$, then we have

$$\dot{e} = -v\theta_e \tag{12}$$

$$\dot{\theta}_e = -\frac{v}{L}\delta \tag{13}$$

and

$$\begin{bmatrix} \dot{e} \\ \dot{\theta}_e \end{bmatrix} = \begin{bmatrix} 0 & -v \\ 0 & 0 \end{bmatrix} \begin{bmatrix} e \\ \theta_e \end{bmatrix} + \begin{bmatrix} 0 \\ -\frac{v}{L} \end{bmatrix} \delta$$

LQR Control

- Now, change the **state** $\mathbf{x}$ to$[e, \theta_e, \delta]^T$ and the **control input** $\mathbf{u}$ to $\dot{\delta}$

Please finish the linear system below:

$$
\begin{aligned}
\mathbf{x}_{k+1} &= A\mathbf{x}_k + B\mathbf{u}_k \\
&= \begin{bmatrix} e_k \\ \theta_{e,k} \\ \delta_k \end{bmatrix} + \begin{bmatrix} \dot{e}_k \\ \dot{\theta}_{e,k} \\ \dot{\delta}_k \end{bmatrix} \Delta t \\
&= \begin{bmatrix} your\ answer & your\ answer & your\ answer \\ your\ answer & your\ answer & your\ answer \\ your\ answer & your\ answer & your\ answer \end{bmatrix} \begin{bmatrix} e_k \\ \theta_{e,k} \\ \delta_k \end{bmatrix} + \begin{bmatrix} your\ answer \\ your\ answer \\ your\ answer \end{bmatrix} [\dot{\delta}_k]
\end{aligned}
$$

Code Structure

```
code/
|-- PathTracking/
        |-- long_controller_*.py
        |-- controller_*.py
        |-- ...
|-- Simulation/
        |-- kinematic_*.py
        |-- simulation_*.py
        |-- ...
|-- tracks/
|-- navigation.py
|-- trajectory_generator.py
|-- ...
```

- long_controller_*.py → Longitudinal Control ↕
- controller_*.py → Lateral Control ↔
- kinematic_*.py → Kinematic Models
- simulation_*.py → Update Simulator
- trajectory_generator.py → Cubic Curve Interpolation, Velocity Profiling, Waypoint Sampling

In `navigation.py`

`__main__` → `navigation()`

Trajectory Planning

Sparse Waypoints (HW1)

Cubic Curve Interpolation

Dense & Smooth Path

Velocity Profiling

Reference Speed

Waypoint Sampling

Trajectory

Path Tracking

Control State u_{t-1}

Update Simulator

State $\zeta_t = [x_t, y_t, \theta_t]^T$

Longitudinal Control ↕

Lateral Control ↔

Control State u_t

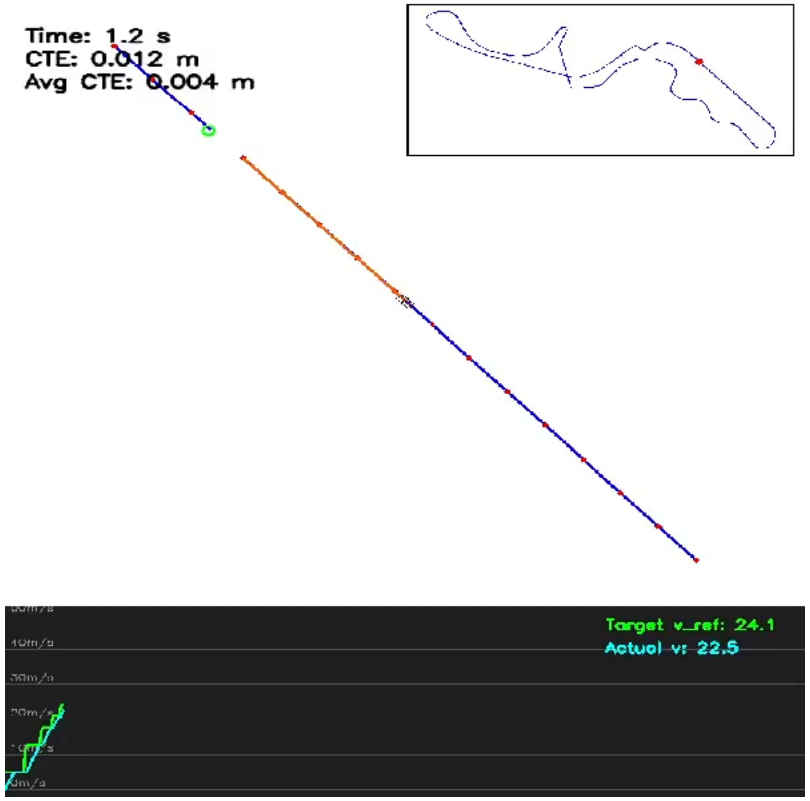
Run Command

```
python navigation.py
  -s [basic/diff_drive/bicycle] (simulator)
  -c [pid/pure_pursuit/stanley/lqr] (controller)
  -t [1000mStraight/400mRunningTrack/Silverstone/Suzuka/Monza] (track)
  -lcs [steering_angle/steering_angular_velocity] (lqr_control_state)
```

- Press ***Esc*** to terminate simulation!

F1 Track Challenge

- You can tune controller parameters, implement additional controllers, or improve any part of the navigation flow to enhance overall system performance.



2x

F1 Track Challenge - Rules

- Do not change the constraints $\boldsymbol{v}_{\mathbf{max}}, \boldsymbol{a}_{\mathbf{lat}}, \boldsymbol{a}_{\mathbf{acc}}, \boldsymbol{a}_{\mathbf{dec}}$ in velocity profiling and any simulator code.
- If you modify any code outside the designated `#TODO` blocks, please mark them with a `#NewFeature` comment, and describe the change in your report.
- Any violation of the above 2 rules will receive 0 points in this challenge!

F1 Track Challenge – Grading Policy

Grading Policy: Let $T_{\text{ref},i}$ be the baseline completion time for track i (**Silverstone: 107s, Suzuka: 115s, Monza: 94s**).

$$S_{T,i} = \max\left(0,\ \min\left(6,\ 4 - k_T(T_i - T_{\text{ref},i})\right)\right)$$
$$S_{E,i} = \max\left(0,\ \min\left(6,\ 4 - k_E(E_i - 1.5)\right)\right)$$
$$\text{Score}_i = \min\left(10,\ S_{T,i} + S_{E,i}\right)$$
$$\text{Final Score} = \left\lceil \frac{1}{3}\sum_i \text{Score}_i \right\rceil$$

Here, $k_T = 0.5$ and $k_E = 2.0$ are penalty coefficients. Note that tracking error is penalized more heavily than completion time, so prioritize accuracy over speed. Try your best!

Submission Deadline

- Same as announcement on NTU COOL (2026/3/29 23:59)
- Hard deadline, no late submission allowed

Submission Rules

- Your written response must be typeset (LaTeX) and in **blue**.
  Also, do not forget to convert it into `report.pdf` before submission.

- Please organize your submission into a folder as follows:

```
[your student ID]_HW2/
└ report.pdf
└ code/
    └ ...
```

  Then, compress this folder into a zip archive named `[your student ID]_HW2.zip` (e.g., `1140123456_HW2.zip`) and submit it to **NTU COOL**.

- Any violation of the above rules will get 10-point penalty

Assignment Checklist

Please check the boxes (☐ → ■) which you have completed

Done	No.	Type	Task	Points
☐	2.1.1	✎	Answer questions (yaw unit, update) about `kinematic_basic.py`	4
☐	2.1.2	✎	List constraints about `simulator_basic.py`	2
☐	2.2.1	✎	List constraints regarding `simulator_different_drive.py`	2
☐	2.2.2	⚙	Complete # TODO 2.2.2 in `navigation.py` for differential drive	4
☐	2.3.1	⚙	Complete `step()` in `KinematicModelBicycle`	5
☐	2.3.2	✎	List constraints regarding `simulator_bicycle.py`	3
☐	3.1	⚙	Complete velocity profiling in `trajectory_generator.py` (Speed limit 4 pts + Smoothing 8 pts)	12
☐	3.2	⚙	Implement Longitudinal PID control in `navigation.py`	8
☐	4.1.1	✎	Identify error term in `PathTracking/controller_pid_basic.py`	2
☐	4.1.2	⚙/✎	Tune PID gains in `controller_pid_basic.py` and describe tuning process	5
☐	4.1.3	⚙/✎	Complete `PathTracking/controller_pid_bicycle.py`	5
☐	4.2.1	⚙	Complete `PathTracking/controller_pure_pursuit_bicycle.py`	8
☐	4.3.1	⚙	Complete `PathTracking/controller_stanley_bicycle.py`	8
☐	4.4.1	⚙	Complete `PathTracking/controller_lqr_bicycle.py`	8
☐	4.4.2	✎	Write down how you choose the weight matrices Q and R	4
☐	4.4.3	✎	Complete linear system of Equation (19)	4
☐	4.4.4	⚙	Complete `PathTracking/controller_lqr_bicycle.py`	6
☐	5.1	⚙/✎	F1 Track Challenge (Choose best model & controller)	10
☐	5.2	✎	Describe attempts to improve performance (Extra)	+5
☐	6.1	✎	(Optional) Provide feedback and suggestions	+5
			Total Score	**100** (+10)

Quick Start

- To setup environment (using conda as example)
  - Navigate to HW2 directory
  - Use the same environment as in HW1
    - `$ conda activate rne_hw1`
    - `$ pip install matplotlib`
- Make a copy of report file on Overleaf
- Follow the report instruction and answer the question one-by-one!
  - All test commands for each problem are listed in the report ☺